

Classes and Objects

class - `__init__` - `self` - Properties - `@classmethod` - `@staticmethod`

```
class Account:
    def __init__(self, cust_id, balance):
        self._cust_id = cust_id
        self._balance = balance
    @property
        # ...
```



Example files

All files are in the `tb_sc/ch_08_classes_and_objects/` folder.

File	Contents
<code>sc_08_01_Car.py</code>	Basic class - Car with <code>__init__</code> and methods
<code>sc_08_02_Account1.py</code>	Account class with getter/setter methods
<code>sc_08_03_use_Account1.py</code>	Client code for Account1
<code>sc_08_04_Account2.py</code>	Account with Transaction log
<code>sc_08_05_use_Account2.py</code>	Client code for Account2
<code>sc_08_06_Account3.py</code>	Account with <code>@property</code> and validation
<code>sc_08_07_use_Account3.py</code>	Client code for Account3
<code>sc_08_08_Account4.py</code>	Account with class variables and <code>@classmethod</code>
<code>sc_08_09_circle_with_op_overload.py</code>	Circle - <code>__eq__</code> , <code>__lt__</code> , <code>__gt__</code>
<code>sc_08_10_overload_getitem.py</code>	Order - <code>__getitem__</code> for indexing and slicing
<code>sc_08_12_bool_truthiness.py</code>	Wallet - <code>__bool__</code> for truthiness
<code>sc_08_15_point_manual.py</code> / <code>_dataclass.py</code>	Point - manual vs <code>@dataclass</code>
<code>sc_08_16_clock.py</code>	Clock class - full date/time with validation
<code>sc_08_17_temperature.py</code>	Temperature - celsius/fahrenheit/kelvin properties

8.1

Creating a class

class - __init__ - self - attributes - methods

First class: Car

```
# file: sc_08_01_Car.py
class Car:
    def __init__(self, brand, color):
        self._brand = brand
        self._color = color

    def drive(self):
        print(f"{self._brand} car is driving!")

    def show_info(self):
        print(f"Car: {self._brand}, Color: {self._color}")

my_car = Car("Toyota", "Blue")
another_car = Car("Volvo", "Red")

my_car.drive()
my_car.show_info()
another_car.show_info()
```

- Class name starts with a capital letter - PascalCase convention
- `__init__()` is the constructor - called when we create an object
- `self` is a reference to the object itself - Python passes it automatically
- Attributes created with `self._name` - leading underscore signals internal use
- Instance methods take `self` as first parameter
- Two objects: each gets its own `_brand` and `_color`

Output

```
Toyota car is driving!
Car: Toyota, Color: Blue
Car: Volvo, Color: Red
```

Class vs object - the difference

CLASS (blueprint)	
<code>_brand</code>	attribute - stores a value
<code>_color</code>	attribute - stores a value
<code>drive()</code>	method - shared code
<code>show_info()</code>	method - shared code

creates



OBJECTS (instances)

- The class defines the structure - like a blueprint or template
- Each object is an independent instance of that class
- Methods (code) are shared between all objects
- Attributes (data) are unique to each object
- **Car** is the class - **my_car** and **another_car** are objects
- We can create as many objects as we want from one class

Analogy

Class = cookie cutter (the mold)

Object = cookie (the result)

All cookies have the same shape.

Each cookie has its own flavor.

`__init__()` - the constructor

```
# __init__() is called automatically on object creation
my_car = Car("Toyota", "Blue")
```

```
# Python rewrites this as:
Car.__init__(my_car, "Toyota", "Blue")
```

```
# Two steps behind the scenes:
# 1. __new__() - allocates memory, creates the object
# 2. __init__() - initialises attributes
```

```
# Always create ALL attributes in __init__():
class Account:
    def __init__(self, cust_id, balance, interest):
        self._cust_id = cust_id    # always here
        self._balance = balance    # always here
        self._interest = interest  # always here
```

```
# NOT scattered across other methods - hard to track,
# risk of AttributeError if method is called first
```

- Called automatically - never call `__init__()` manually
- Receives the same arguments we pass to the class name
- Ensures the object is in a valid state from the start
- All fixed attributes belong here - not in other methods

vs C++ / Java

In C++ and Java, fields are declared in the class body before they are used.

In Python, attributes are created at runtime via `self._name`.

This gives flexibility - but also responsibility: keep them in `__init__`.

What is self?

```
# Code in a class is shared by all objects.
# Attributes are unique to each object.

# When we write:
my_car.drive()

# Python rewrites it as:
Car.drive(my_car)    # self = my_car

# Same at creation:
my_car = Car("Toyota", "Blue")
# Becomes:
Car.__init__(my_car, "Toyota", "Blue")

# But my_car doesn't exist yet...?
# Python does it in two steps:
# 1. __new__() allocates memory -> object exists
# 2. __init__() is called -> self points to it
```

- **self** is always the first parameter in instance methods
- Python passes **self** automatically - we never pass it manually
- **self** always refers to the specific object the method is called on
- `my_car.drive()` and `Car.drive(my_car)` are identical
- This is how the same code works differently for different objects

Under the hood

```
my_car.brand = 'Toyota'
another_car.brand = 'Volvo'
```

`drive()` is one function in memory.
`self` tells it WHICH object's
`_brand` to use.

`self` = the object 'running' the
method right now.

Dot notation and when to use a class

```
# Dot notation: object.attribute or object.method()
my_car._brand          # read attribute
my_car.drive()         # call method
my_car.show_info()     # call method

# Create a class when:
# - something has BOTH data and behaviour that belong together
# - you need multiple instances with the same structure
# - you want to organise code cleanly and reusably

# A function is enough when:
def calculate_area(radius):
    return 3.14 * radius ** 2

# No data to keep between calls -> no need for a class
```

- Dot . tells Python: use something belonging to this object
- Without the dot, Python does not know which object we mean
- Ask: does it have both data AND behaviour that go together?
- If yes - good candidate for a class
- If it just does one calculation with no state - use a function

Good class candidates

BankAccount - balance, transactions,
deposit(), withdraw()

Car - brand, color, speed,
drive(), brake()

Student - name, grades,
average(), is_passing()

8.2

A more advanced class: Account

Getters/setters - __str__ - Transaction - Properties

Basic Account class

```
# file: sc_08_02_Account1.py
class Account:
    def __init__(self, cust_id, account_no, start_balance,
interest):
        self._cust_id      = cust_id
        self._account_no   = account_no
        self._balance      = start_balance
        self._interest     = interest

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
        return self._balance

    def withdraw(self, amount):
        if amount <= self._balance:
            self._balance -= amount
        return self._balance

    def __str__(self):
        return (f"Customer id   = {self._cust_id}\n"
                f"Balance          = {self._balance:.2f}")
```

- Four attributes created in `__init__()` - all with leading underscore
- `deposit()` validates positive amount, `withdraw()` checks sufficient balance
- Both methods return the new balance
- `__str__()` called implicitly by `print()` - without it we get a memory address
- Interest stored as integer: 5 means 5%

Python has no private

`account._balance` # works!

Python does not enforce private.

Leading `_` is a convention:

'stay away - use the methods'.

It is trust between developers,
not a technical barrier.

__str__() and name mangling

```
# __str__() is called implicitly by print()
print(account1)          # calls account1.__str__()
account1.__str__()        # explicit - rarely done

# Without __str__(): ugly memory address
# <__main__.Account object at 0x000001E3B5C0>

# Try: comment out __str__() and run print(account1)

# Name mangling with double underscore:
class Account:
    def __init__(self):
        self.__balance = 0    # double underscore

# Python renames it internally to:
# _Account__balance
# Still accessible if you know the name!
acc = Account()
print(acc._Account__balance)  # works
```

- `print(object)` automatically calls `object.__str__()`
- If missing, Python falls back to the base class - gives a memory address
- Name mangling with `__name` (double underscore): Python renames to `_ClassName__name`
- Purpose: avoid name conflicts in inheritance - NOT true privacy
- Recommendation: use single underscore `_name` as the standard convention

Under the hood

`print(object)` calls:
`object.__str__()`

Ultimate base class object
has `__str__()` too - returns
the memory address string.

Our `__str__()` overrides it.

Account with Transaction log

```
# file: sc_08_04_Account2.py
from datetime import datetime

class Transaction:
    def __init__(self, amount, trans_type):
        self._amount      = amount
        self._trans_type  = trans_type
        self._timestamp   = datetime.now()

    def __str__(self):
        return (f"{self._timestamp:%Y-%m-%d %H:%M:%S} | "
                f"{self._trans_type.capitalize():8} | "
                f"{self._amount:8.2f}")

class Account:
    def __init__(self, ...):
        ... # same as before
        self._transactions = [] # NEW

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            self._transactions.append(
                Transaction(amount, "deposit")) # NEW
        return self._balance
```

- Two classes collaborating: Account creates and stores Transaction objects
- Transaction is deliberately simple - stores data, knows how to print itself
- Account's responsibility: create transactions and keep the list
- `print(trans)` calls `Transaction.__str__()` implicitly
- Each deposit, withdrawal and interest addition creates one Transaction

Output

```
2026-05-07 21:38:42 | Deposit | 2000.00
2026-05-07 21:38:42 | Withdraw | 500.00
2026-05-07 21:38:42 | Interest | 300.42
```

8.3

Properties

@property - @name.setter - validation - read-only

Properties - the Pythonic way

```
# file: sc_08_06_Account3.py (extract)
class Account:
    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, new_balance):
        if new_balance < 0:
            print("Balance cannot be negative!")
        else:
            self._balance = new_balance

# Client code uses clean names:
account1 = Account(123, 456, 1000, 2.5)
print(account1.balance)    # getter called
account1.balance = 2000    # setter called
account1.balance = -500    # "Balance cannot be negative!"
```

- Client code uses `account1.balance` - no parentheses, no underscore
- `@property` decorates the getter - called when name appears on the RIGHT of =
- `@balance.setter` decorates the setter - called when name appears on the LEFT of =
- Internally the class still uses `self._balance`
- The setter can contain validation without the client code knowing

Fact: what is a decorator?

`@property` and `@balance.setter` start with @ above a method.

`@property` makes Python treat the method as an attribute - so we write `account1.balance` not `account1.balance()`.

Recipe for a property

```
# The property recipe - three parts:

class MyClass:
    def __init__(self, value):
        self._value = value    # internal storage

    @property                    # 1. getter
    def value(self):
        return self._value

    @value.setter                # 2. setter
    def value(self, new_value):
        if new_value < 0:
            raise ValueError("Must be positive")
        self._value = new_value

# Read-only property: skip the setter
@property
def double(self):
    return self._value * 2    # computed, no setter
```

- Attribute stored with underscore: `self._value`
- Getter with `@property` - read with `obj.value`
- Setter with `@value.setter` - set with `obj.value = x`
- Read-only: define only the getter, omit the setter
- Computed property: return a value calculated from other attributes

When to use `@property`

- Validate or restrict what can be set
- Hide internal implementation detail
- Make an attribute read-only
- Return a computed value

Don't use it for simple attributes with no extra logic - plain attributes are simpler.

Property example: Temperature

```
# file: sc_08_17_temperature.py
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius    # uses setter -> validated

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError(f"Below absolute zero: {value}")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9 / 5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self.celsius = (value - 32) * 5 / 9    # reuses celsius setter

    @property
    def kelvin(self):
        # read-only - no setter
        return self._celsius + 273.15
```

- One internal value: `_celsius` - but three interfaces externally
- Constructor writes `self.celsius = ...` - runs the setter, validates on creation
- Fahrenheit setter calls `self.celsius = ...` - reuses validation for free
- `kelvin` is read-only: no setter, uniquely determined by Celsius
- Client picks whichever scale is natural - the class handles conversions

Elegant detail

`self.celsius = celsius`
in `__init__` uses the SETTER.
Validation happens at creation.

`fahrenheit` setter calls
`self.celsius = ...`
-> `celsius` validation runs
automatically. No duplication.

8.4

dataclass

When the class is just data

@dataclass - less boilerplate

```
# Manual version:
class Point:
    def __init__(self, x, y):
        self._x = x
        self._y = y
    def __str__(self):
        return f"Point(x={self._x}, y={self._y})"
    def __eq__(self, other):
        return self._x == other._x and self._y == other._y

# ---- Same result with @dataclass: ----
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int

p1 = Point(3, 4)
p2 = Point(3, 4)
print(p1)           # Point(x=3, y=4)
print(p1 == p2)     # True
```

- `@dataclass` generates `__init__()`, `__repr__()` and `__eq__()` automatically
- Fields declared as `name: type` directly in the class block
- Plain attributes - no leading underscore, no properties
- Default values: `x: int = 0`
- Not a replacement for `Account` - that class needs validation and full logic

Nice to know: type hints

`@dataclass` requires annotated fields: `x: int` is not optional.

This makes `@dataclass` a good argument for using type hints consistently - we get functionality for free as a reward.

Chapter 8 Part 1 - Key takeaways

- A class is a blueprint; an object is a concrete instance of that class
- `__init__()` is the constructor - called automatically, creates all attributes
- `self` is a reference to the current object - Python passes it automatically
- `my_car.drive()` and `Car.drive(my_car)` are exactly the same thing
- Leading underscore `_name` signals 'internal use' - Python does not enforce it
- `__str__()` gives readable output via `print()`; without it you get a memory address
- `@property`: getter called on read, `@name.setter` called on write - validation for free
- In `__init__`, write `self.name = value` (not `self._name`) to trigger the setter
- Fahrenheit setter calls `self.celsius = ...`, reusing validation without duplication
- `@dataclass` generates `__init__`, `__repr__` and `__eq__` automatically for data classes
- When something has both data and behaviour that belong together - consider a class

